

Southwest  
Tech

# Python Programming Foundations

Week 4 - Debugging, Testing, & Reading Structured Code  
Day 1: "Debugging as Evidence Gathering"

1

---

---

---

---

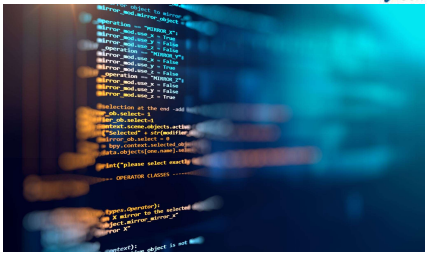
---

---

---

---

Southwest  
Tech



## Review Coding Lab

2

---

---

---

---

---

---

---

---

Southwest  
Tech

## A Bug Is Information

A bug is information, not a verdict.

When code breaks or gives the wrong answer, the program is giving you clues.

Today, practice debugging by asking:

- ▶ What did I expect?
- ▶ What actually happened?
- ▶ Where is the first useful clue?
- ▶ What fix does the evidence justify?



3

---

---

---

---

---

---

---

---




bug -> information

A “bug” is “information”

4

---

---

---

---

---

---

---

---



## Build Becomes Inspect



In earlier weeks, you built small programs.  
Now you inspect behavior when a program does not do what it should.

Debugging uses familiar skills:

- reading code
- running code
- tracing values
- explaining output

5

---

---

---

---

---

---

---

---



## Symptoms Show Up, Sources Hide

A *symptom* is what you notice first - The *source* is where the problem actually begins.

Examples:

- ▶ **symptom**: wrong final total
- ▶ **source**: tax calculated from the wrong value
- ▶ **symptom**: program will not run
- ▶ **source**: missing quote, colon, or parenthesis



6

---

---

---

---

---

---

---

---

Southwest Tech

The visible problem may start earlier in the program.

**SYMPTOM**

What we see

The program ran. Here is the result:

**Total: -25** ← Wrong

The output is wrong, but the cause is not here.

**SOURCE**

Where it starts

Earlier in the program...

```

1 total = 0
2 for n in numbers:
3     value = n - 5 ← Bug here (off by 10)
4     total = total + value
5     print("Total:", total)

```

A small mistake here changes everything earlier.

Trace backward. Find the true source.

**Symptoms Show Up, Sources Hide**

7

---

---

---

---

---

---

---

---

Southwest Tech

**What Counts as Success Today**

Working baseline -> compare structure -> revise one meaningful thing -> explain the improvement.

A useful revision should make the program:

|         |                |                  |                   |
|---------|----------------|------------------|-------------------|
| clearer | easier to test | easier to change | easier to explain |
|---------|----------------|------------------|-------------------|

8

---

---

---

---

---

---

---

---

**What Counts as Success Today**

- compare two approaches
- collect timing evidence
- describe growth cautiously
- use basic Big-O vocabulary
- identify a limitation of your evidence

9

---

---

---

---

---

---

---

---



## What We Will Use Today

- ▶ expected versus actual
- ▶ tracebacks
- ▶ labeled print-debugging
- ▶ one-change-at-a-time repair
- ▶ simple evidence notes

*These tools help you investigate before changing code.*

10

---

---

---

---

---

---

---

---

---

---



## Debugging Toolbox

Small tools. Clear thinking. Better programs.

- expected vs actual**  
Define what should happen. Compare with what did.  
Expected: 42  
Actual: -2  
RuntimeError: IndexError
- traceback clue**  
Use the error message to find the failing point.  
Traceback (most recent call last):  
 File "main.py", line 17, in <module>  
 total = total + num  
TypeError: unsupported operand type(s) for +: 'int' and 'str'
- labeled print**  
Print key values with labels to see what's happening.  
INPUT n = 5  
STEP value = -3  
STEP total = -2
- one-change repair**  
Make one small change. Test again. Confirm the fix.  
Change:  
value = n + 5  
(was n - 5)
- evidence notes**  
Record what you observed and what you changed.  
Observed: total went negative.  
Changed line 17.  
Now correct.

Debugging is not guessing. It's collecting clues, making small changes, and learning.

11

---

---

---

---

---

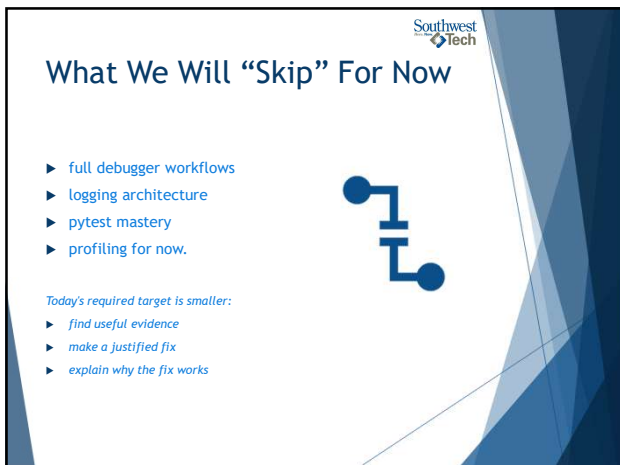
---

---

---

---

---



## What We Will "Skip" For Now

- ▶ full debugger workflows
- ▶ logging architecture
- ▶ pytest mastery
- ▶ profiling for now.

*Today's required target is smaller:*

- ▶ find useful evidence
- ▶ make a justified fix
- ▶ explain why the fix works

12

---

---

---

---

---

---

---

---

---

---



### Parked for Later

Powerful topics. Useful later. Not our focus today.

**full debugger workflows**  
Step through, inspect, and control complex programs.

**logging architecture**  
Designing logs that are structured, consistent, and useful at scale.

**pytest mastery**  
Fixtures, parametrization, mocks, and advanced testing patterns.

**profiling**  
Measure performance, find bottlenecks, improve efficiency.

We'll come back to these when you're ready. Strong foundations first.

## Parked For Later

13

---

---

---

---

---

---

---

---



### Expected Versus Actual



Expected behavior is what should happen.  
Actual behavior is what happened when the program ran.

"Debugging" begins when you compare:

- ▶ expected output
- ▶ actual output
- ▶ the difference between them

14

---

---

---

---

---

---

---

---



**Expected**  
**Result: 42**  
This is what we planned to see.

**Actual**  
**Result: -7**  
This is what we actually saw.

compare before changing code.

## Expected vs. Actual

15

---

---

---

---

---

---

---

---



16

---

---

---

---

---

---

---

---



17

---

---

---

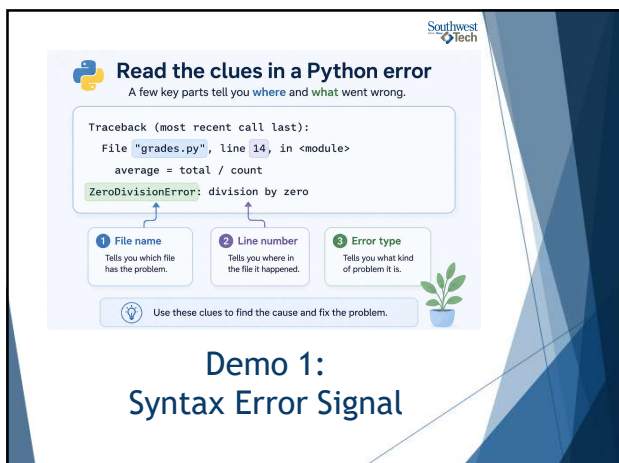
---

---

---

---

---



18

---

---

---

---

---

---

---

---



19

---

---

---

---

---

---

---

---



20

---

---

---

---

---

---

---

---



21

---

---

---

---

---

---

---

---

Southwest  
Tech

## Ask One Question, Print One Clue

Print-debugging should answer a specific question.

**Weak**

```
Code_Sample.py
1
2 print("here")
3
```

**Better**

```
2
3 print("subtotal before tax:", subtotal)
4
```

Every debug print should reveal a value or path you need to inspect.

22

---

---

---

---

---

---

---

---

Southwest  
Tech

**Weak Print Debugging**  
Doesn't tell you what's happening.

**Code**

```
print("here")
```

**Output**

```
here
here
here
```

*Not helpful. You have to guess.*

**Why it's weak**

- You don't know what value this is, or where it came from.

→

**Stronger Print Debugging**  
Answers a question.

**Code**

```
print("subtotal before tax:", subtotal)
```

**Output**

```
subtotal before tax: 87.58
```

*Clear and useful. You learn something.*

**Why it's stronger**

- Tells you what the value is.
- Shows the variable name or context.
- Helps you understand what's happening.

**Good rule: Make your prints say something useful.** If someone else saw this output, would they understand it?

## Ask One Question, Print One Clue

23

---

---

---

---

---

---

---

---

**Demo #3**

"Grade Summary Debugging"

24

---

---

---

---

---

---

---

---





## What to Watch For

Watch where the grade summary first goes wrong.

Trace:

- ▶ starting student
- ▶ running total
- ▶ added score
- ▶ calculated average
- ▶ first incorrect value

25

---

---

---

---

---

---

---

---



## Grade Calculation Trace

Follow the data as it moves through each checkpoint.

| Student | Running Total (Before Adding) | Added Score (This Entry) | Average (After Adding) |
|---------|-------------------------------|--------------------------|------------------------|
| Alex    | 0<br>(no scores yet)          | 88<br>(Test 1)           | 88.00<br>(88 ÷ 1)      |
| Alex    | 88<br>(from above)            | 92<br>(Test 2)           | 90.00<br>(180 ÷ 2)     |
| Alex    | 180<br>(from above)           | 76<br>(Test 3)           | 109.33<br>(328 ÷ 3)    |

**Investigate Here**  
This is the first wrong average. Check this row.

**How to read this trace:**  
Each row shows how one score affects the running total and the new average. Find the first suspicious value, then check the calculation at that step.

**Tip:** The first wrong value is usually closest to the real cause.

## Demo 3: Grade Summary Debugging

26

---

---

---

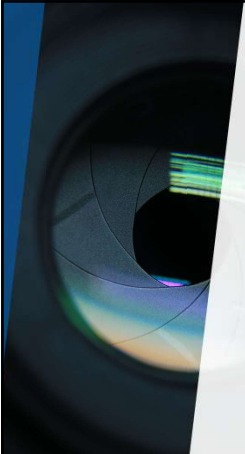
---

---

---

---

---



## Demo #4

"Order Total Debugging"

27

---

---

---

---

---

---

---

---



## What to Watch For

Watch the calculation stages.  
Useful checkpoints:

- ▶ line total
- ▶ subtotal
- ▶ discount
- ▶ tax
- ▶ final total

*The bug is not just the wrong final number. The signal is the first wrong step.*

28

---

---

---

---

---

---

---

---



## Calculation Stages

Follow each checkpoint. The wrong value is first used in **Stage 3**.

| Scenario              | 1 Line Total      | 2 Subtotal                        | 3 Discount                                          | 4 Tax                                       | 5 Final Total              |
|-----------------------|-------------------|-----------------------------------|-----------------------------------------------------|---------------------------------------------|----------------------------|
| Items total: \$200.00 | \$200.00          | \$200.00                          | \$220.00                                            | \$17.60                                     | \$237.60                   |
| Discount: 10%         | Sum of all items. | Same as line total (no fees yet). | Should subtract 10% of \$200 (\$20.00), not add it. | 8% of \$220.00 (Based on the wrong amount). | Amount due (after tax).    |
| Tax rate: 8%          | Looks good        | Looks good                        | Wrong value first used here                         | Carries error forward                       | Final result is also wrong |

**How to use this:**  
Check each stage in order. The first stage with a wrong value is usually where the bug is. Fix that stage, and the later stages should fix themselves.

**Investigate Here**  
The discount should be a subtraction, not an addition.

## Demo 4: Order Total Debugging

29

---

---

---

---

---

---

---

---



## Evidence Before Repair

Good debugging evidence narrows the problem.

Evidence may be:

- ▶ a traceback line
- ▶ expected versus actual output
- ▶ a labeled debug print
- ▶ the first wrong value
- ▶ a simple check after the fix

*Do not change several things before you know what the evidence says.*

30

---

---

---

---

---

---

---

---



## Common Failure: Changing Code First

Changing code before observing can destroy the trail.

Instead:

- ▶ reproduce the problem
- ▶ record what happened
- ▶ inspect one clue
- ▶ change one thing
- ▶ run again

31

---

---

---

---

---

---

---

---



## A Simple, Focused Debugging Loop

One step at a time. Small changes. Clear answers.

**1 OBSERVE**

Look closely at what happened.



Ask yourself:

- What do I see?
- What was I expecting?
- What's the actual result?

**2 INSPECT ONE CLUE**

Pick one clue and examine it closely.



Ask yourself:

- Which value looks suspicious?
- Which step might be the cause?
- What does this clue tell me?

**3 CHANGE ONE THING**

Make a small, targeted change.



Ask yourself:

- What is the best guess?
- What small change can I try?
- Can I change just one thing?

**4 RUN AGAIN**

Run the program and see what changed.



Ask yourself:

- Did the result change?
- Did it fix it?
- What did I learn?
- If not, what will I try next?

**Key idea:** Slow down. Use clues. Make one change. Learn from each run.

**This process helps you find the real cause, step by step.**

## Common Failure: Changing Code First

32

---

---

---

---

---

---

---

---



## Assignment 6

For Assignment 6, repair broken code and explain the debugging process.

Your work should show:

- ▶ the issue you found
- ▶ the evidence that helped
- ▶ the fix you made
- ▶ the check that proves the fix works

33

---

---

---

---

---

---

---

---



### DEBUGGING NOTES

Capture the facts. Make one change. Verify the result.

- 1 EXPECTED**  
What do you expect to happen?
- 2 ACTUAL**  
What actually happened?
- 3 EVIDENCE**  
What output, message, or value shows what happened?
- 4 FIX**  
What one change will you make?
- 5 CHECK AFTER FIX**  
What do you expect to see now? How will you verify it works?

## Debugging Notes Template

34

---

---

---

---

---

---

---

---

### Evidence For A Debugging Submission

Useful debugging evidence includes:

- ▶ corrected '.py' file
- ▶ expected-vs-actual comparison
- ▶ labeled debug output
- ▶ traceback clue
- ▶ simple check or test case
- ▶ explanation of why the fix works

The evidence should support the repair, not just decorate the submission.

35

---

---

---

---

---

---

---

---



### DEBUGGING EVIDENCE

One change. Clear evidence. Problem understood.

- 1 Corrected File: order\_total.py**

```

1 def calculate_total(subtotal, discount_rate, tax_rate):
2     discount = subtotal * discount_rate
3     after_discount = subtotal - discount # FIXED
4     tax = after_discount * tax_rate
5     total = after_discount + tax
6     return total
7
8 # Example run
9 total = calculate_total(100, 0.20, 0.10)
10 print(f'Total: {total:.2f}')
```

Change made: subtract discount instead of adding it.
- 2 Expected vs Actual**

| EXPECTED | VS | ACTUAL (before fix) |
|----------|----|---------------------|
| 80.00    |    | 120.00              |

Correct total after 20% discount and 10% tax.

This happened when the discount was added instead of subtracted.
- 3 Labeled Debug Output (before fix)**

```

subtotal: 100.00
discount_rate: 0.20
discount: 20.00
after_discount: 80.00
tax: 8.00
total: 88.00
```

These prints show the first suspicious value.
- 4 Traceback Clue**

```

Traceback (most recent call last):
  File "order_total.py", line 10, in calculate_total
    after_discount = subtotal + discount
TypeError: unsupported operand type(s) for +: 'float' and 'int'
```

Clear line 3 is where the discount operation happens.
- 5 Explanation & Fix**

The discount was added instead of subtracted on line 3. This made the after\_discount too large, which caused the tax and total to be too large as well.

Fix: Change 'subtotal + discount' to 'subtotal - discount'.

## Evidence

36

---

---

---

---

---

---

---

---



### Closing Success Check

If you found the first useful signal, you are debugging.

Before submitting Assignment #6, ask:

- ▶ What was wrong?
- ▶ What evidence showed it?
- ▶ What did I change?
- ▶ How did I check the fix?
- ▶ Can I explain the repair without guessing?



37

---

---

---

---

---

---

---

---



## Thank you!

## Have Fun!

38

---

---

---

---

---

---

---

---